

地理空间数据库 2025-2026学年秋
冬学期空间数据库设计汇报

暖爪寻踪——基于LBS 的流浪动物保护一体化 数据库

成员

顾子琰、陈音琪

时间

2025/12/29



目录

01

需求分析

02

概念设计

03

逻辑设计

04

物理设计

05

数据库实施



01

需求分析

当前，流浪动物数量呈现逐年递增趋势，城市中流浪猫、流浪狗等面临着食物匮乏、疾病侵袭、意外伤害等生存困境，同时也可能引发公共卫生安全隐患、城市环境破坏等社会问题。

01 流浪动物现状与社会救助需求

激增的动物数量与不足的社会救助能力

当前，流浪动物数量呈现**逐年递增趋势**，城市中流浪猫、流浪狗等动物遍布社区、公园、街头等公共区域，面临着**食物匮乏、疾病侵袭、意外伤害等生存困境**，同时也可能引发**公共卫生安全隐患**（如携带狂犬病病毒、传播寄生虫）、**城市环境破坏**等社会问题。

01 救助信息不对称

爱心人士发现流浪动物后，难以快速对接专业救助机构或志愿者，导致救助响应滞后。

02 普通宠物店笼养的存在安全隐患

不同区域的救助点、收容所信息不互通，无法实现资源统筹调配，部分区域救助力量过剩而部分区域存在救助盲区。

03 救助效率低下

传统救助依赖人工巡查发现流浪动物，耗时耗力且覆盖范围有限，难以精准定位需要紧急救助的动物位置。

04 社会公众参与救助的渠道不够便捷

多数人因缺乏有效信息和参与途径，难以切实参与到流浪动物保护工作中，进一步加剧了救助工作的被动性。

02 LBS技术的应用价值

实现救助线索精准定位与快速流转

当用户发现流浪动物后，可通过终端设备自动获取当前地理位置坐标并上传至系统，救助机构或志愿者能通过系统快速查看流浪动物的具体位置，规划最优救助路线，大幅缩短救助响应时间。

构建救助资源可视化网络

系统可基于LBS技术展示区域内救助点、收容所、志愿者的分布位置，实现救助资源的实时统筹调配，避免资源浪费，同时方便用户快速查找附近的救助资源

提升领养与失宠招领的匹配效率

对于可领养动物，用户可通过位置筛选功能查找附近的领养资源，降低领养成本；对于失宠宠物，主人上传走失位置后，系统可推送匹配的招领信息给附近用户，或让用户按位置范围查询相关信息，提高宠物找回概率。



LBS技术以地理位置信息为核心，结合**移动互联网、数据库存储**等技术，能够精准破解当前流浪动物救助工作中的**信息不对称和效率低下**问题。

03 项目梗概

- **核心目标：**构建一套支持流浪动物**信息上传、领养申请、订阅、失宠招领**的LBS数据库系统。
- **具体目标：**提升流浪动物**救助效率**、降低**领养门槛**、实现**救助信息实时互通**、助力走失宠物找回。

整合救助资源

优化领养流程

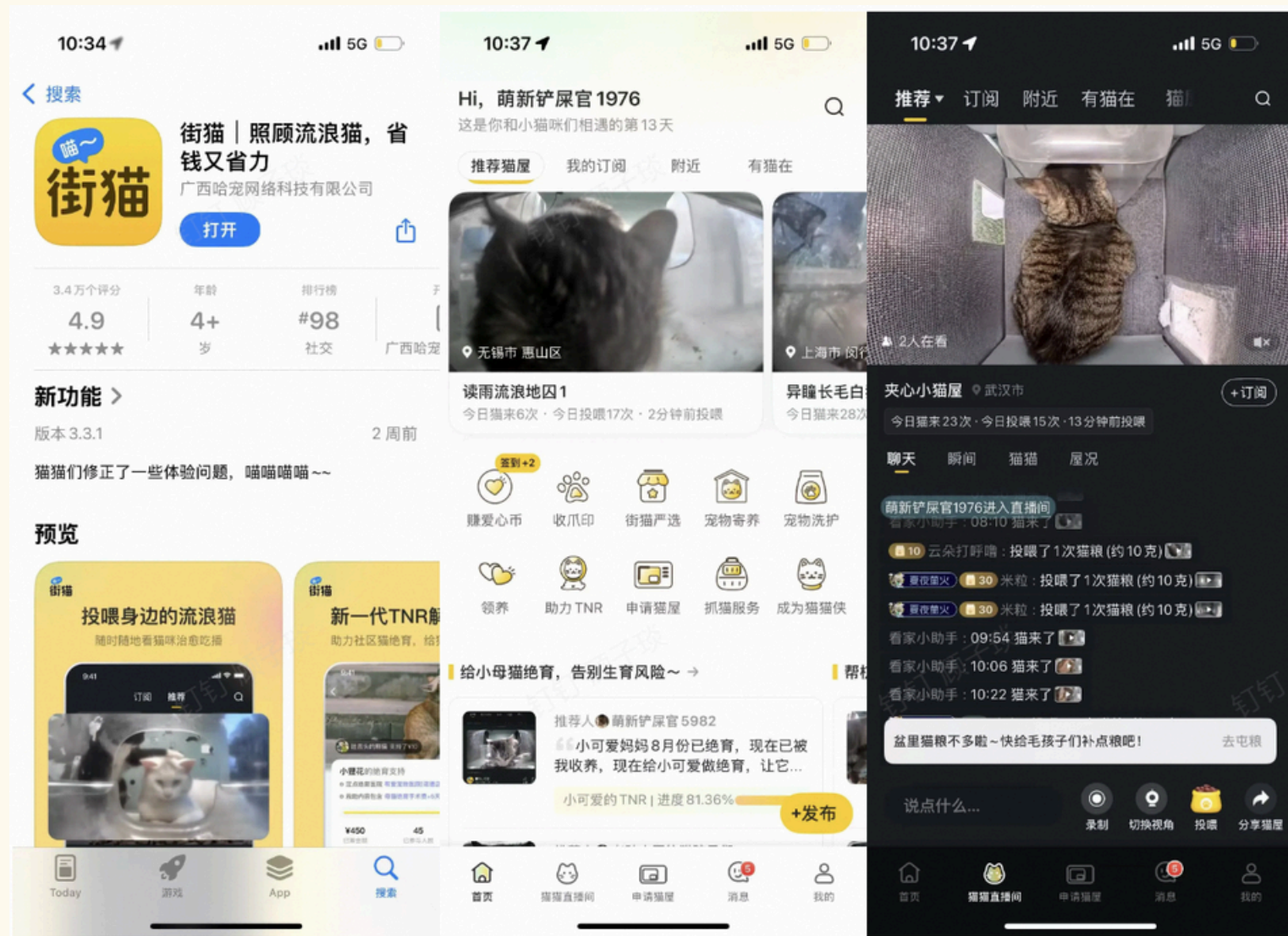
搭建信息互助平台

提升救助效率

04 需求调研

国内：街猫-流浪猫云喂养绝育领养平台

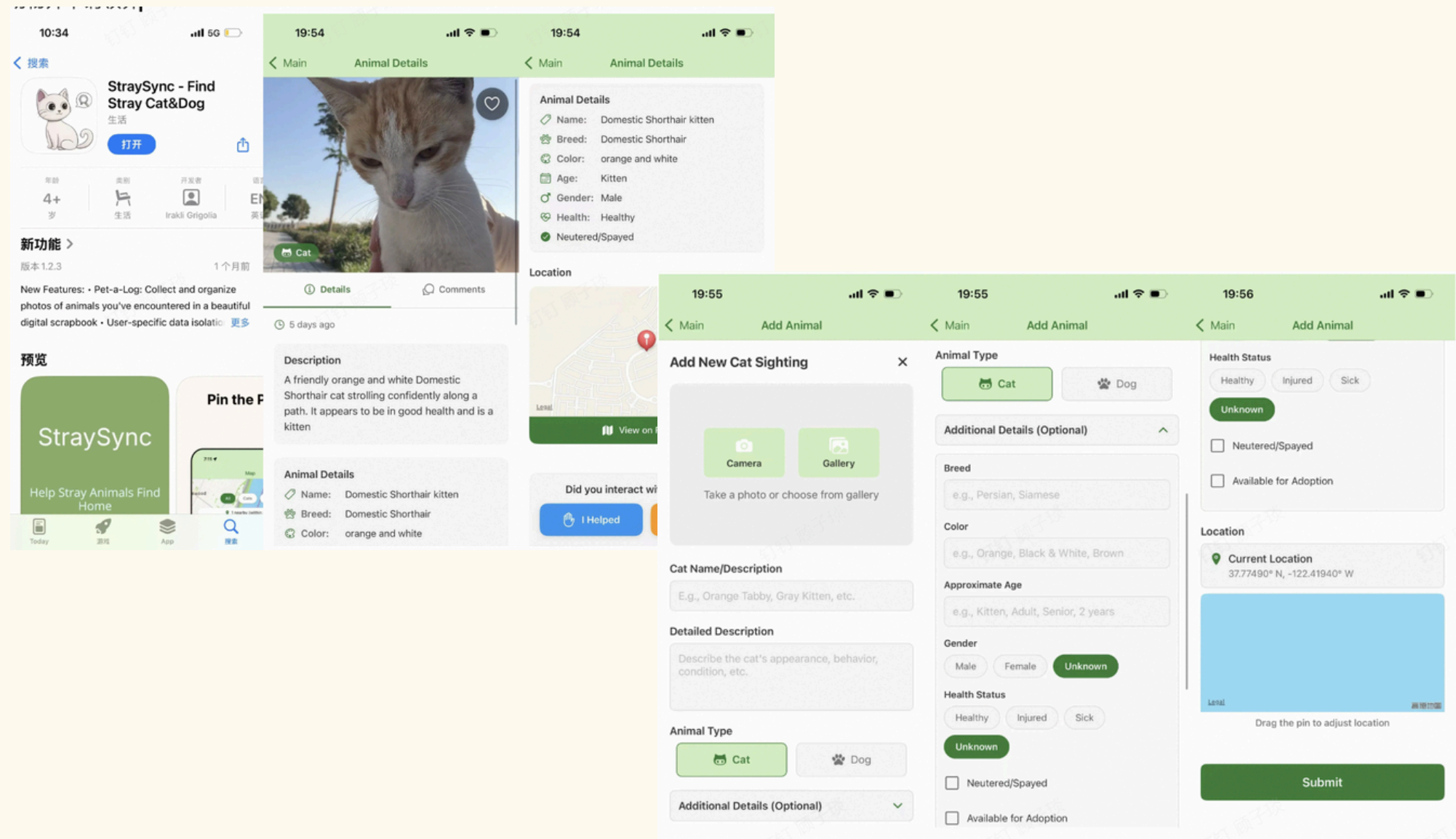
主要实现的功能：用户订阅并助力喂养小猫/绝育申请与资金筹备/动物救助/宠物服务



调研当今市场上现存的流浪动物保护软件

国外：StraySync-Find stray cats&dogs

主要实现功能：用户遇到流浪猫拍照上传并会显示位置与描述（details）/用户申请Lost&Found失宠招领/筛选流浪动物并申请领养



04 需求调研

01 建议：自由上传动物照片，形成**动物主页**；定位功能只对动保同学开发，不让所有人知道动物定位（**模糊准确动物位置**）



访谈浙江大学动物保护协会志愿者同学

02 将用户分为不同种类（用户us下增加属性role），给不同种类用户不同的权限，分享地点的用户的信息只有动保用户可以看；动保成员有认证组织，可以进行审核。



05 用户需求

我们将用户分为四类，并定义其对静态和动态数据的需求。

普通用户

- 流浪动物的发现者或潜在领养人。
- 权限：Select (除精确位置以外的Animal表信息), Insert (发现、报失、领养)。

动保管理员

- 数据表管理员/审核员，负责线索核实、领养审批和数据维护。
- 权限：Select, Insert, Update, Delete (全表)。

宠物失主

- 发布寻宠启示、系统推荐的匹配动物

系统/数据库管理员

- 维护空间索引和性能。
- 管理数据库空间结构等。



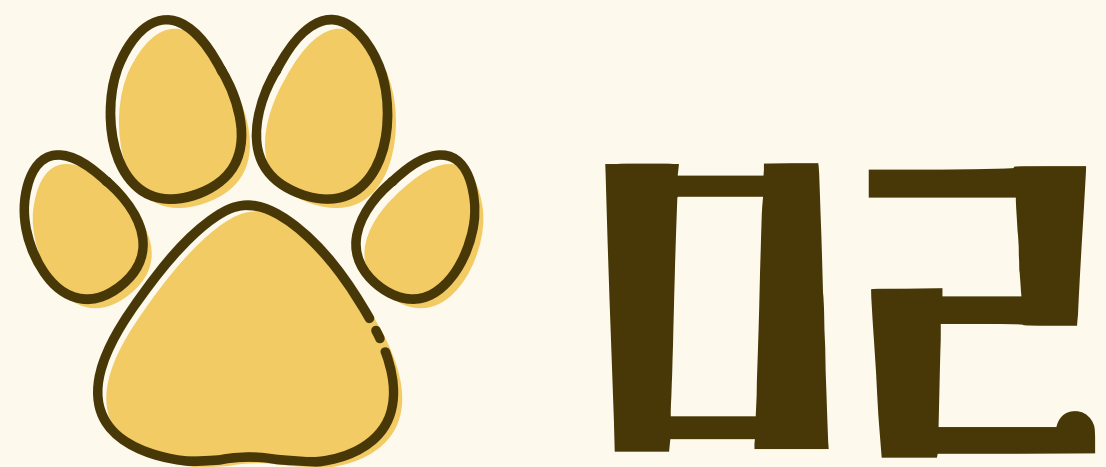
06 数据类型与存储方式

数据类型

- **静态空间数据**：救助设施 (Facility) 的位置。坐标固定，不随时间变化。
- **动态时空数据**：动物位置轨迹 (Discovery/Animal Location)。记录动物在不同时间出现在不同地点的历史轨迹。

存储形式

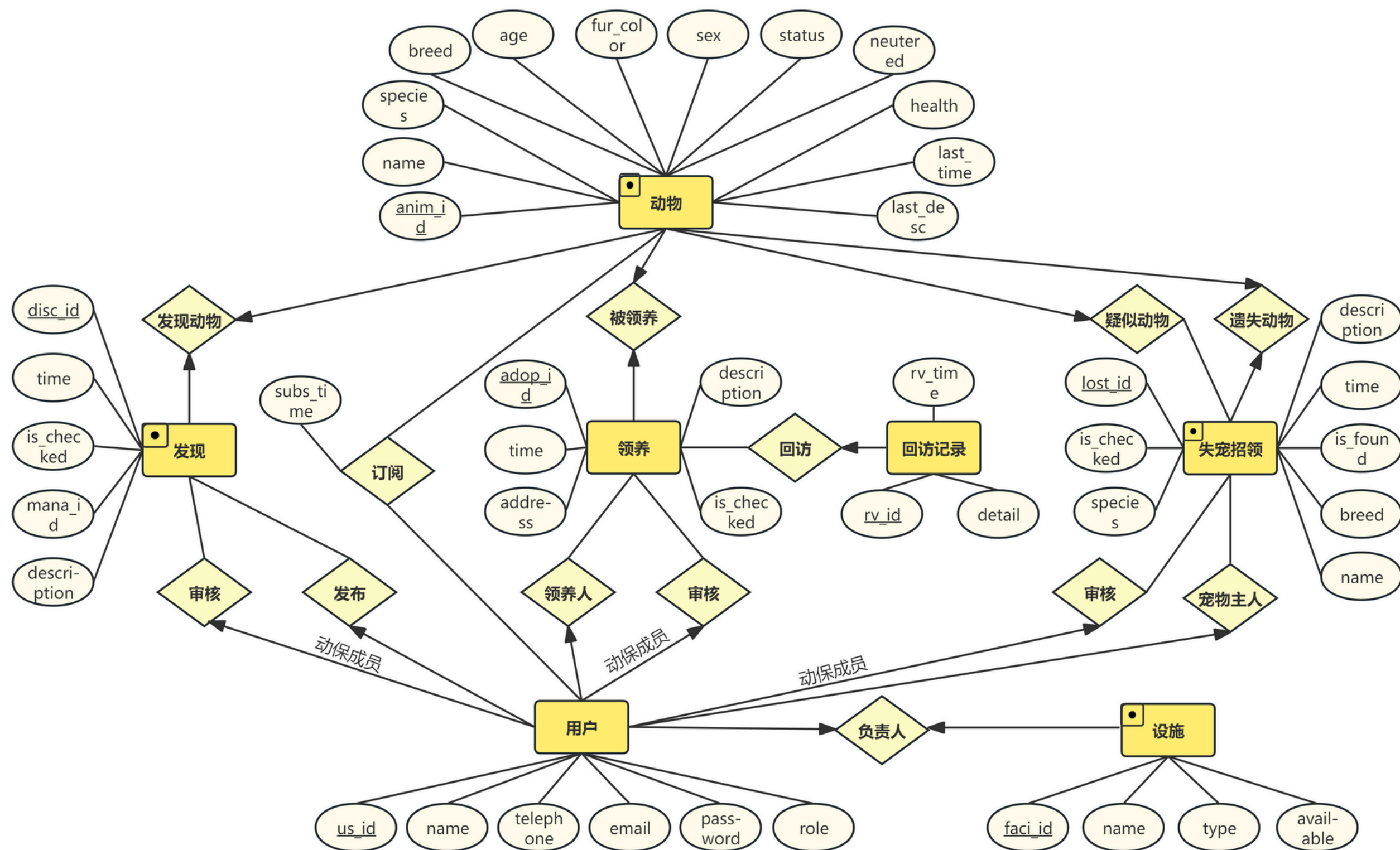
- **空间属性**：使用 GEOMETRY(Point, 4326) 存储经纬度。
- **时间属性**：使用 TIMESTAMP 记录发现或丢失的具体时刻。
- **静态数据**：使用B+树进行存储。



概念设计

基于用户需求，分析该应用的实体、实体属性和实体之间的联系，绘制空间扩展E/R图，描述应用场景的约束条件，实体的空间属性使用象形图表示。

01 空间扩展E/R图



02 实体和实体属性

动物 (Animal)

- id (唯一性, 主码)
- 动物名字 name
- 种类 species
- 品种 breed
- 年龄 age
- 颜色 fur color
- 性别 sex
- 状态 status
- 健康状况 health
- 是否绝育 neuter
- 上次被目击时间 last_time
- 上次被目击描述 last_desc
- 上次被目击地址 location(geom(point))

用户 (Us)

- id (唯一性, 主码)
- 用户名 name
- 电话 telephone
- 邮箱 email
- 密码 password
- 角色 role

发现 (Discovery)

- id (唯一性, 主码)
- 描述 dsscription
- 发现时间 time
- 审核状态 is_checked
- 发现地点 location(geom(point))

设施 (facility)

- id (唯一性, 主码)
- 设施名 name
- 设施地址 location(geom(point))
- 设施种类 type
- 现在可否提供帮助 available

领养(Adoption)

- id (唯一性, 主码)
- 领养时间 time
- 领养地址 address
- 审核状态 is_checked
- 领养理由 description

失宠招领(Lost_Report)

- id (唯一性, 主码)
- 丢失地点 last_seen (geom (point))
- 丢失时间 time
- 描述情况 description
- 名字 name
- 种类 species
- 品种 breed
- 状态 (是/否) is_found
- 审核状态 is_checked

回访记录(rv_record)

- id (唯一性, 主码)
- 回访时间 rv_time
- 回访细节 detail

03 关系概念设计

- 用户+动物：订阅 (N to N)
- 用户+发现：(普通用户) 发布 (1 to N)
(动保成员) 审核 (1 to N)
- 动物+发现：获取动物活动地点 (1 to N)
- 用户+失宠招领：(普通用户) 宠物主人 (1 to N)
(动保成员) 审核 (1 to N)
- 动物+失宠招领：遗失动物 (1 to 1)
疑似动物 (1 to N)
- 用户+领养：(普通用户) 发布 (1 to N)
(动保成员) 审核 (1 to N)
- 设施+用户：设施所属人 (1 to 1)



04 约束条件

- **时空一致性：**动物的“最近位置”必须由最新的“发现事件”触发更新。
- **数据类型：**明确各字段数据类型（字符型、数值型、日期型、坐标型、二进制等）
- **长度限制：**如用户名、密码、联系方式的字符长度限制
- **非空约束：**如用户ID、动物ID、定位坐标等核心字段非空
- **唯一性约束：**如用户ID、申请ID、动物ID唯一
- **取值范围：**如审核状态（待审核/通过/拒绝）、动物状态（待救助/已救助/已领养）
- **用户权限：**不同用户类型拥有不同权限（普通用户/动保成员/收容所负责人/数据库管理员），
在进行操作前需要进行ROLE检验



03

逻辑设计

基于空间扩展的E/R图，转换为关系和函数依赖，进行相应的关系分解，减少设计异常，但不能严重降低数据插入和查询效率，影响用户体验。

用户表(Us)

Us (us_id, name, password, telephone, email, role)

FD: us_id → {name, password, telephone, email, role}

- us_id: int, SERIAL PRIMARY KEY
- name: TEXT NOT NULL
- password: varchar(8) NOT NULL
- telephone: TEXT
- email: TEXT
- role TEXT CHECK (role IN ('common_user', 'protection_user', 'main_manager', 'shelfter_host'))

动物表(Animal)

Animal (anim_id, name, species, breed, age, fur_color, sex, status, health, neutered, last_time, last_desc, location(geom:Point))

FD: anim_id → {name, species, breed, age, fur_color, sex, status, health, neutered, last_time, last_desc, location}

- anim_id: SERIAL PRIMARY KEY
- name: TEXT
- species: TEXT CHECK
(species IN ('猫', '狗', 'Other'))
- breed: TEXT
- age: FLOAT (单位: 岁)
- fur_color: TEXT
- status: TEXT{流浪/收容/遗失/已领养/已找回}
- health: INT (0: 不健康, 1: 健康)
- neutered: INT (0: 未绝育, 1: 已绝育)
- last_time: TIMESTAMP
- last_desc: TEXT
- location: geometry(Point, 4326)
- sex: TEXT CHECK (sex IN ('male', 'female'))

发现记录表(Discovery)

Discovery (disc_id, time, is_checked, *mana_id*, description, *anim_id*, *us_id*, location(geom:Point))

外码: mana_id, anim_id, us_id

FD: disc_id → {time, is_checked, mana_id, description, anim_id, us_id, location}

- disc_id: iSERIAL PRIMARY KEY
- time: TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
- mana_id: INT REFERENCES Us(Us_id)
- description: TEXT
- anim_id: INT REFERENCES Animal(anim_id)
- us_id: INT REFERENCES Us(Us_id),
- is_checked: INT {0:未审批, -1:未通过, 1:通过}
- location:geometry(Point, 4326)

失宠招领表(Lost_Report)

Lost_Report (lost_id, *us_id*, *anim_id*, time, last_seen(geom:Point), is_found,description, *mana_id*, is_checked, name, species, breed, stray_id)

外码: mana_id, anim_id, us_id, stray_id

FD: lost_id → {us_id, anim_id, time, last_seen, is_found,mana_id, is_checked}

- lost_id: serial primary key
- us_id: int
- anim_id: int
- time: timestamp
- last_seen: geometry(Point, 4326)
- is_found: INT (0: 未找到, 1: 已找到)
- description: text
- is_checked: INT
(0:未审批 1: 已审批 2: 有疑似动物 3: 审核通过)
- stray_id: INT REFERENCES Animal(anim_id)

收养表(Adoption)

Adoption (adop_id, *us_id*, *anim_id*, time, address, *mana_id*, is_checked)

外码: mana_id, anim_id, us_id

FD: adop_id → {us_id, anim_id, time, address, mana_id, is_checked}

- adop_id: SERIAL PRIMARY KEY
- us_id: INT REFERENCES Us(us_id)
- anim_id: INT REFERENCES Animal(anim_id)
- time: TIMESTAMP DEFAULT CURRENT_TIMESTAMP
- address: TEXT
- mana_id: INT REFERENCES Us(us_id)
- is_checked: INT (-1、 0、 1)

回访记录表

RV_Record(rv_id, rv_time, detail, *adop_id*)

外码: adop_id

FD: rv_id → {rv_time, detail, adop_id}

- rv_id: serial primary key
- rv_time: timestamp
- detail: text
- adop_id: int, REFERENCES Adoption(adop_id)

设施表

Facility (faci_id, name, type, available, location(geom:Point), *us_id*)

外码: us_id

FD: facility_id → {name, type, available, location, us_id}

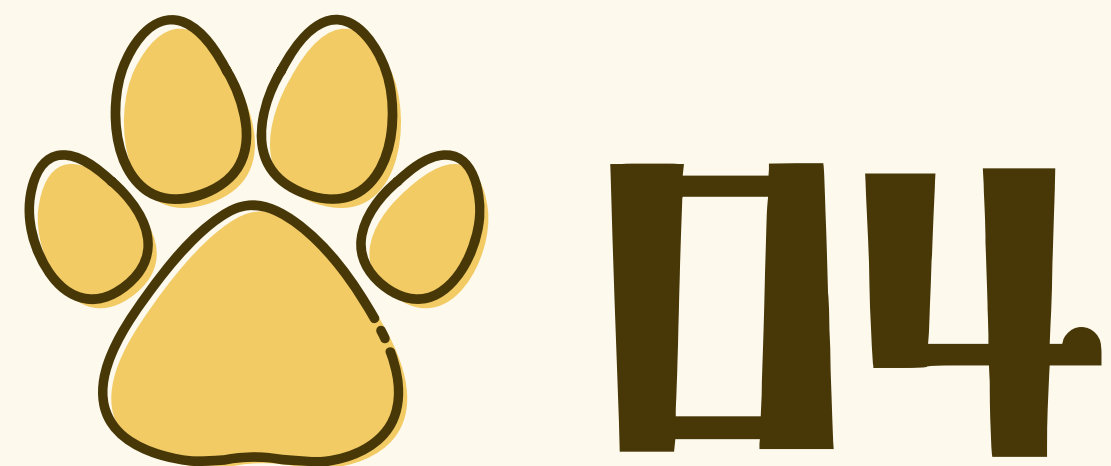
- faci_id:serial primary key
- name:text
- type: text "宠物医院/宠物店/猫咖/狗咖/其他收容所"
- available: 0:暂不提供帮助 1: 可以提供帮助
- location: geometry(Point, 4326)

订阅表

Subscription(us_id, animal_id, subs_time)

FD: us_id, animal_id $\rightarrow$ {subs_time}

- us_id: int,
- animal_id: int,
- subs_time: timestamp
- primary_key(us_id, animal_id)



物理设计

分析用户需求的性能要求，描述高峰期用户数据库访问量。针对哪些常见查询创建索引，针对哪些需求创建视图，针对哪些约束创建触发器。分析每类用户的权限，包括选择、插入、更新和删除等操作，保障数据的安全性

数据库实现具体功能

1. 普通用户 (Common User)

- 领养与互动：可以通过视图查看待领养动物，并提交领养申请，申请会自动关联当前用户信息。
- 流浪动物发现：发现流浪动物后，可提交地理位置、时间及描述信息进行上报。
- 寻宠报案：当宠物丢失时可在线报案，系统会自动记录失主身份并将其列入寻宠数据库。
- 智能匹配与认领：系统会自动计算 10km 范围内与其丢失宠物特征相符的流浪动物，支持失主发起认领申请。
- 个性化订阅：支持对特定动物进行订阅，实时跟踪其状态变化，并可管理自己的订阅列表。

2. 动保成员 (Protection User)

- 档案全权管理：拥有对用户、动物及相关业务表的完整管理权限。
- 领养审批：审核普通用户的领养申请，通过后系统会自动更新动物状态为“已领养”并记录审核人。
- 发现验证与建档：验证用户上报的发现记录，审核通过时可选择更新已有档案或自动为新动物建立档案。
- 报案审核与档案迁移：审核寻宠报案并自动建档；当认领成功时，负责触发档案合并流程，将丢失宠物的临时档案与流浪动物的正式档案进行迁移合并。
- 回访监督：负责记录并查看领养后的回访情况，确保动物福利。

3. 收容所负责人 (Shelter Host)

- 收容设施维护：管理自己名下的收容所或宠物医院信息。
- 状态实时调控：可随时修改设施的营运状态（如“营运中”或“暂停服务”）以及地理位置信息。

01 性能与安全性

性能要求与索引

高峰访问量：

假设救助活动期间，每秒产生 50 次“查询周围流浪动物”的请求（空间查询）。

空间索引：

针对 **Animal.status** 使用 R 树空间索引加速查询，针对 **Animal.location**、**Lost_Report.last_seen** 和 **Facility.location** 创建 GiST 索引。覆盖了系统中最频繁的三个操作：状态筛选、身份校验（主键）和地理位置匹配。

创建视图加速查询

```
1 %%sql
2 CREATE INDEX idx_animal_status ON Animal USING BTREE (status);
3 CREATE INDEX idx_animal_location ON Animal USING GIST (location);
4 CREATE INDEX idx_lost_seen_location ON Lost_Report USING GIST (last_seen);
5 CREATE INDEX idx_facility_location ON Facility USING GIST (location);
```

安全性分析

1. 访问控制与权限管理

通过划分不同的角色并为特定用户分配权限，建立了 **common_user**（普通用户）、**protection_user**（动保成员）、**shelter_host**（收容所负责人）和 **main_manager**（数据库管理员）四种角色。

2. 视图层隔离

通过视图层过滤敏感数据并进行信息屏蔽：例如 **View_User_Adoption_Submit** 视图移除了 **is_checked**（审核状态）字段，防止用户在提交申请时尝试伪造审核通过的状态。

3. 触发器与业务逻辑安全

利用 **INSTEAD OF** 和 **BEFORE** 触发器防止非法数据的注入。在所有提交类触发器中，系统不依赖用户输入的 ID，而是通过 **SELECT us_id INTO v_us_id FROM Us WHERE name = SESSION_USER** 强制绑定当前会话用户。这有效杜绝了“冒充他人 ID 提交数据”的风险。

02 视图分析

- **1. 面向普通用户的视图 (Common User) :**
 - 待领养动物公示视图 (View_Wait_Adoption) : 展示所有状态为“流浪”或“收容”且健康状况良好的动物，供用户浏览领养。
 - 领养申请提交视图 (View_User_Adoption_Submit) : 允许用户提交领养申请，并查看自己申请的审核状态。
 - 我的关注动物动态视图 (View_My_Subscriptions) : 核心业务视图。展示当前登录用户订阅的动物动态，并通过窗口函数 (COUNT(*) OVER) 实时显示该用户的总订阅数。
 - 个人订阅管理视图 (View_My_Pet_Matches) : 利用空间函数 ST_DWithin，自动为失主匹配方圆 10 公里内物种相符的流浪动物。
 - 周边疑似宠匹配视图 (Discovery_commonuser) : 允许普通用户提交发现流浪动物的线索 (时间、地点、描述) 。
 - 失主认领申请视图 (Lost_Report_Owner) : 允许失主发布寻宠启事，记录丢失位置和时间。
- **2. 面向动保组织成员的视图 (Protection Member)**
 - 目击线索管理视图 (View_Discovery_Manage) : 用于审核用户提交的发现线索，动保成员可以将线索关联到现有动物 ID 或创建新档案。
 - 领养回访追踪视图 (View_Adoption_FollowUp) : 展示已领养动物的历史记录和回访明细，确保后续福利。
- **3. 面向收容所/医院老板的视图 (Shelter Host)**
 - 救助设施运营管理视图 (View_Shelter_Management) : 允许收容所或医院老板管理名下设施的名称、精确位置 (WKT格式) 及当前的营运状态。

02 视图分析

```
1  %%sql
2  -- 创建领养回访记录视图
3  DROP VIEW IF EXISTS View_Adoption_FollowUp;
4
5  CREATE VIEW View_Adoption_FollowUp AS
6  SELECT
7      a.anim_id,
8      a.name AS animal_name,
9      a.species,
10     u.name AS adopter_name,
11     ad.time AS adoption_date,
12     rv.rv_time AS visit_time,
13     rv.detail AS visit_detail
14 FROM Adoption ad
15 JOIN Animal a ON ad.anim_id = a.anim_id
16 JOIN Us u ON ad.us_id = u.us_id
17 LEFT JOIN RV_Record rv ON ad.adop_id = rv.adop_id
18 WHERE ad.is_checked = 1; -- 仅查看已审核通过的领养回访
19
20 -- 授权
21 GRANT SELECT ON View_Adoption_FollowUp TO protection_user;
22 GRANT ALL PRIVILEGES ON TABLE rv_record TO protection_user;
23 GRANT USAGE, SELECT, UPDATE ON SEQUENCE rv_record_rv_id_seq TO protection_user;
```

03 触发器分析

- **1. 领养管理模块：**通过触发器实现从“用户申请”到“管理员审核”的完整闭环。
- **trig_handle_adoption_submission**（视图 View_User_Adoption_Submit）
- **实现功能：**实现“伪插入”。当普通用户在视图中提交领养申请时，触发器自动获取当前登录的 SESSION_USER 对应的 ID，校验该动物是否处于可领养状态，最后将数据插入到实际的 Adoption 表中。
- **trg_protect_is_checked**（表 Adoption）
- **实现功能：**状态联动。当管理员（动保成员）将申请状态修改为“通过”（is_checked = 1）时，自动将 Animal 表中对应的动物状态更新为“已领养”，并记录审核员 ID。
- **2. 动物发现模块 (Discovery Module)：**用于处理用户发现流浪动物后的报送与系统建档。
- **View_User_Discovery_Trig**（视图 Discovery_commonuser）
- **核心功能：**自动关联用户。将普通用户提交的发现记录插入 Discovery 表，并自动填入当前操作者的 us_id。
- **trg_discovery_verify**（表 Discovery）
- **核心功能：**自动建档或更新。审核通过后，如果该发现指向已知动物，则更新其位置和状态；如果是新发现的动物，则在 Animal 表中自动插入一条新记录。

03 触发器分析

- **3. 寻宠/报案模块 (Lost Report Module)：涉及临时档案与正式档案的合并。**
- **insert_lostreport_trig (视图 Lost_Report_Owner)**
- **实现功能：失主报案入口。将失主填写的丢失信息存入 Lost_Report 表，并自动绑定当前用户 ID。**
- **trg_lost_report_verify (表 Lost_Report)**
- **实现功能：自动建立“遗失”档案。当报案被审核通过时，系统自动在 Animal 表创建一个状态为“遗失”的临时档案，便于后续匹配。**
- **trg_view_claim_submit (视图 View_Claim_Submit)**
- **实现功能：认领关联。允许失主在发现系统匹配的流浪动物后，提交认领申请，将报案记录与流浪动物档案关联，并进入待审核状态。**
- **trg_final_merge_prepare (BEFORE) 与 trg_final_merge_cleanup (AFTER) (表 Lost_Report)**
- **实现功能：双触发器协同完成档案合并。**
- **Prepare：在审核找回前，将报案时的“临时档案”信息迁移到“正式流浪档案”中，更新状态为“已找回”，并迁移关注该动物的所有订阅记录。**
- **Cleanup：在更新完成后，安全删除已失去意义的临时档案记录。**

03 触发器分析

- 4. 个人功能与设施管理
- trg_manage_my_subs（视图 View_Manage_My_Subs）
- 实现功能：订阅管理。通过视图简化操作，用户只需在视图中执行插入或删除，触发器便会自动处理 Subscription 表与当前用户 ID 的绑定工作。
- trg_shelter_management（视图 View_Shelter_Management）
- 实现功能：设施运维。允许收容所负责人管理自己的设施，包括新增设施和修改现有设施的营运状态（通过坐标转换等逻辑）。

03 触发器分析

- 4. 个人功能与设施管理
- trg_manage_my_subs（视图 View_Manage_My_Subs）
- 实现功能：订阅管理。通过视图简化操作，用户只需在视图中执行插入或删除，触发器便会自动处理 Subscription 表与当前用户 ID 的绑定工作。
- trg_shelter_management（视图 View_Shelter_Management）
- 实现功能：设施运维。允许收容所负责人管理自己的设施，包括新增设施和修改现有设施的营运状态（通过坐标转换等逻辑）。

03 触发器分析

```
%%sql
-- 创建一个触发器，当新插入的 Discovery 记录被检查时，自动更新 Animal 表中的信息
CREATE OR REPLACE FUNCTION func_handle_discovery_verification()
RETURNS TRIGGER AS $$
DECLARE
    new_anim_id INT;
    v_us_id INTEGER;
BEGIN
    SELECT us_id INTO v_us_id FROM Us WHERE name = SESSION_USER;
    IF (NEW.is_checked = 1 AND OLD.is_checked = 0) THEN
        -- 情况 A: 动保成员辨认后输入了已存在的 animal_id
        IF NEW.anim_id IS NOT NULL THEN
            UPDATE Animal
            SET
                last_time = NEW.time,
                location = NEW.location,
                last_desc = NEW.description,
                status = '流浪'
            WHERE anim_id = NEW.anim_id
            AND (last_time IS NULL OR NEW.time > last_time);
            NEW.mana_id = v_us_id;

            -- 情况 B: 动保成员确认是新动物 (anim_id 为空)
        ELSE
            INSERT INTO Animal (species, status, last_time, last_desc, location)
            VALUES ('Other', '流浪', NEW.time, NEW.description, NEW.location)
            RETURNING anim_id INTO new_anim_id;
            NEW.anim_id := new_anim_id;
            NEW.mana_id = v_us_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_discovery_verify ON Discovery;
CREATE TRIGGER trg_discovery_verify
BEFORE UPDATE ON Discovery
FOR EACH ROW
EXECUTE FUNCTION func_handle_discovery_verification();
```


04 权限管理

普通用户

role=Common_user

- select Animal的具体信息（位置信息需要经过特殊保护，即模糊具体经纬度）；
- insert 发现动物的最新位置
- insert领养申请，update疑似动物；
- insert失宠招领。

收容所负责人

role=Shelfter_host

普通用户权限+

- insert收容所
- update更新自己收容的所有属性

动保成员

role=Protection_user

- select, delete, insert所有表（可以查看动物的精准位置信息）
- 审核领养申请、失宠招领

数据库管理员

role=main_manager

最高权限

04 权限管理

```
1  %%sql
2  -- 基础表权限
3  GRANT SELECT ON TABLE Us TO common_user;
4  REVOKE ALL ON TABLE Animal FROM common_user;
5  GRANT ALL PRIVILEGES ON TABLE Us TO protection_user;
6  GRANT ALL PRIVILEGES ON TABLE Animal TO protection_user;
7  GRANT SELECT ON Us TO shelfter_host;
8
9  GRANT USAGE, SELECT, UPDATE ON SEQUENCE animal_anim_id_seq TO protection_user;
```

05 应用需求相关搜索

查询30天内未回访动物

▷ ▾

```
1 %%sql
2 -- 查询最近一次回访超过30天或领养满30天从未回访的动物
3 SELECT
4     a.name AS animal_name,
5     u.name AS adopter_name,
6     ad.time AS adoption_date,
7     COALESCE(MAX(rv.rv_time), ad.time) AS last_interaction_time,
8     (CURRENT_DATE - COALESCE(MAX(rv.rv_time), ad.time)::date) AS days_since_last_check
9 FROM Adoption ad
10 JOIN Animal a ON ad.anim_id = a.anim_id
11 JOIN Us u ON ad.us_id = u.us_id
12 LEFT JOIN RV_Record rv ON ad.adop_id = rv.adop_id
13 WHERE ad.is_checked = 1 -- 领养已通过
14 GROUP BY a.name, u.name, ad.time, ad.adop_id
15 HAVING COALESCE(MAX(rv.rv_time), ad.time) < NOW() - INTERVAL '30 days';
```

[7]

...

* [postgresql://lisi:***@localhost:5432/project](#)

1 rows affected.

...

animal_name	adopter_name	adoption_date	last_interaction_time	days_since_last_check
小花	zhangsan	2025-09-30 10:46:41.206391	2025-11-29 10:47:20.766702	30

05 应用需求相关搜索

查询7天内流浪动物出现热点区域

```
1  %%sql
2  -- 将几何数据投影到平面坐标系 (SRID: 3857) 来进行 500 米半径的聚类, 计算流浪动物出现热点
3  WITH Cluster_Results AS (
4      SELECT
5          disc_id,
6          description,
7          location,
8          ST_ClusterDBSCAN(ST_Transform(location, 3857), eps := 500, minpoints := 2) OVER () AS cluster_id
9      FROM Discovery
10 )
11 SELECT
12     cluster_id,
13     COUNT(*) AS count,
14     ST_AsText(ST_Centroid(ST_Collect(location))) AS centerpoint,
15     STRING_AGG(description, ' | ') AS total_des
16 FROM Cluster_Results
17 WHERE cluster_id IS NOT NULL
18 GROUP BY cluster_id
19 ORDER BY count DESC;
```

[]

... * postgresql://lisi:***@localhost:5432/project

1 rows affected.

...

cluster_id	count	centerpoint	total_des
0	2	POINT(120.08949999999999 30.305)	蓝田寝室楼下看到那只萨摩耶 (大白) 在转悠 在农生环大楼后面发现一只受惊的哈士奇



05

数据库实施

请见演示内容。



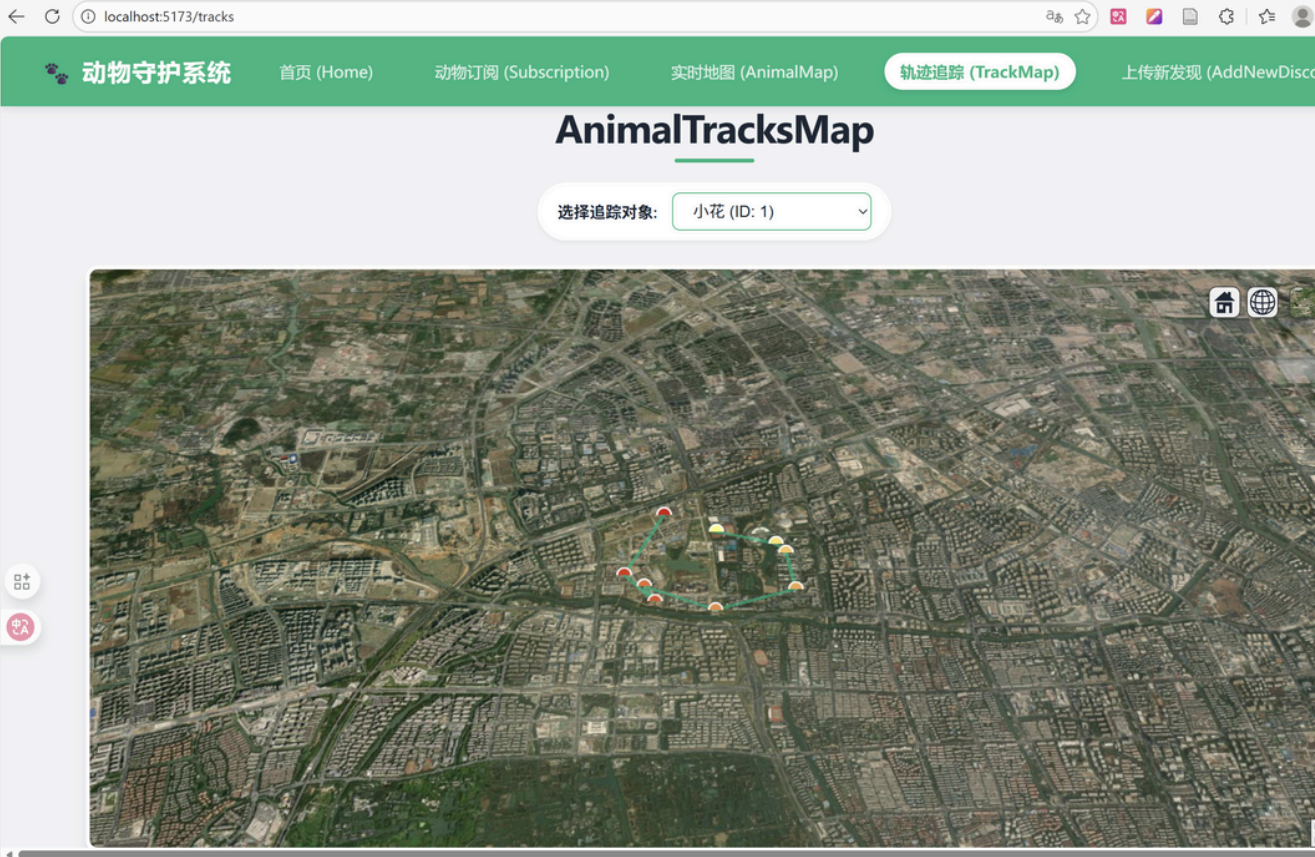
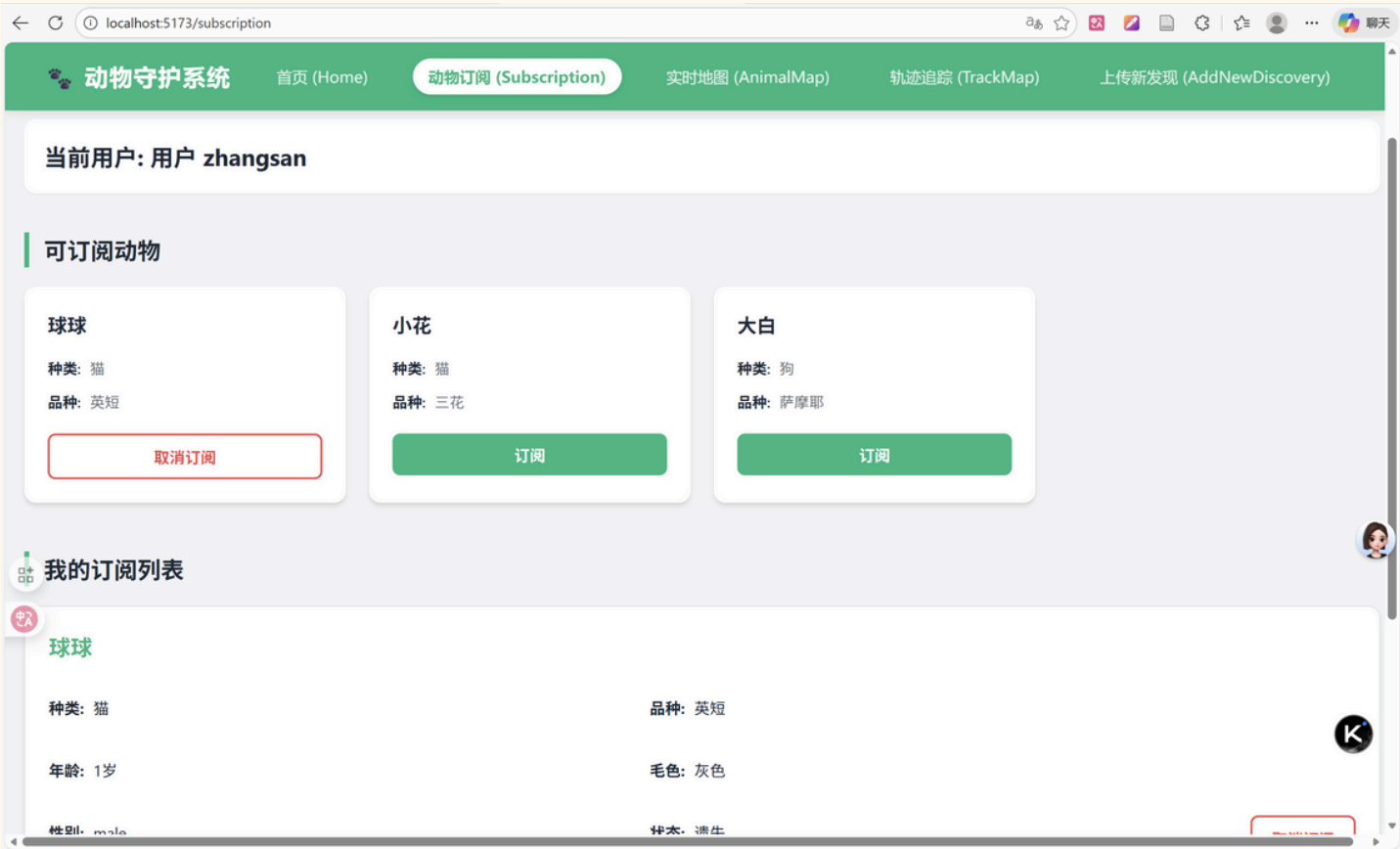
咕

应用原型实现

请见演示内容。

实现形式

- 实现形式：Web网页端应用开发（本地模式）
- 前端框架：React+Vite框架
- 后端框架：Express.js框架
- 地图框架：Cesium
- 实现语言：Javascript+内嵌SQL



地理空间数据库 2025-2026学年秋
冬学期空间数据库设计汇报

感谢观看！
恳请批评指正！

成员

顾子琰、陈音琪

时间

2025/12/29

